UAV Software Engineering: ROS 2 Architecture

Custom Interfaces, QoS, and Flight-Critical Orchestration

1 Introduction to Custom Interfaces (IDL)

In standard software development, passing data between functions is handled via memory pointers or pass-by-value structs. In a distributed UAV architecture, nodes may be running on different CPU cores, different Docker containers, or entirely different physical hardware (e.g., a Companion Computer talking to a Ground Control Station).

ROS 2 solves this distributed data serialization problem using an Interface Definition Language (IDL). Rather than writing raw C++ structs, developers define message schemas in language-agnostic `.msg` or `.srv` files. During the `colcon build` process, a code generator (specifically `rosidl_default_generators`) translates these schemas into highly optimized C++, Python, and DDS-specific IDL data structures.

2 Designing a UAV Telemetry Message

While ROS 2 provides standard types (e.g., `sensor_msgs/msg/NavSatFix`), specialized drone operations often require custom telemetry payloads that pack multiple data points into a single, bandwidth-efficient packet.

Consider the following definition for `DroneTelemetry.msg`:

```
std_msgs/Header header
float64 latitude
float64 longitude
float32 altitude_amsl_m
float32 battery_voltage_v
uint8 flight_mode
bool is_armed
```

When a C++ node publishes this custom message, the ROS 2 middleware (RMW) serializes the object into a Common Data Representation (CDR) byte stream. CDR handles Endianness automatically, ensuring that a Ground Control Station on an x86_64 Windows machine can seamlessly decode a packet sent by an ARM64 companion computer on the drone.

3 DDS Quality of Service (QoS): The Aerospace Lifeline

The most significant architectural upgrade from ROS 1 to ROS 2 is the underlying Data Distribution Service (DDS) middleware. Standard TCP/IP networking guarantees delivery but suffers from unbounded latency and head-of-line blocking—both unacceptable in real-time flight control. DDS operates over UDP, providing deterministic, tunable communication through **Quality of Service (QoS)** policies.

In UAV engineering, especially when operating over lossy Radio Frequency (RF) datalinks, tuning QoS is a regulatory necessity.

3.1 Reliability Policy

The Reliability policy dictates whether the middleware guarantees packet delivery.

- `RELIABLE`: The middleware mandates acknowledgment (ACK) from the subscriber. If a packet is lost, a NACK is sent, and the publisher retransmits it. *Aerospace Use Case:* Critical commands such as "Arm Motors", "Deploy Parachute", or "Upload Waypoint Mission". Missing these packets is catastrophic.

- `BEST_EFFORT`: The middleware sends the packet once (fire-and-forget). No ACKs are tracked, saving immense network overhead. *Aerospace Use Case:* High-frequency sensor streams (e.g., 100Hz IMU). If a quadcopter misses a 10-millisecond-old IMU reading, retransmitting it is worse than useless, as the drone has already rotated. It is better to process the next fresh packet immediately.

3.2 Durability Policy

Durability determines how the middleware handles "late joiners"—nodes that subscribe to a topic after the publisher has already started sending data.

- `VOLATILE`: Messages are not saved. A late subscriber only sees messages published *after* it connects.
- `TRANSIENT_LOCAL`: The publisher keeps a history of the most recent messages and immediately fires them to any new subscriber that joins the graph. *Aerospace Use Case:* The "Flight Mode" state (e.g., `STABILIZED`, `AUTO`). If a Ground Station reconnects mid-flight, it must instantly know the current mode, rather than waiting for the drone to manually broadcast it again.

4 Advanced QoS: Deadlines and Liveliness (Failsafe Triggers)

Modern UAV architectures leverage advanced DDS features to build inherent safety mechanisms directly into the middleware, circumventing the need for custom "watchdog" threads.

4.1 Deadline and Liveliness Policies

The `DEADLINE` policy enforces a strict contract: a publisher promises to send a message at least once every T milliseconds. If the publisher fails to meet this deadline (due to a software crash or network loss), ROS 2 automatically fires a `RequestedDeadlineMissed` callback on the subscriber. The flight controller can use this hardware-level trigger to instantly engage Return-To-Launch (RTL).

Similarly, the `LIVELINESS` policy monitors the heartbeat of a node. Using `MANUAL_BY_TOPIC`, the publisher must explicitly assert its liveliness, proving not just that the process is running, but that the specific sensor loop hasn't deadlocked.

5 DO-178C Inspired Lifecycle Management

Standard ROS 2 nodes begin executing their internal logic immediately upon instantiation. In mission-critical UAV operations (e.g., pre-arm checks, sensor calibration), unmanaged initialization introduces unpredictable race conditions.

To mitigate this, industrial robotics relies on the **Lifecycle Node** model (`rclcpp_lifecycle::LifecycleNode`), mirroring finite state machine (FSM) paradigms required for DO-178C compliant aviation software:

1. **Unconfigured:** The node is instantiated, but hardware interfaces are dormant.
2. **Inactive (Configured):** Memory is allocated via `on_configure()`. Publishers exist but are muted (network transmission disabled).
3. **Active:** Triggered via `on_activate()`, timers begin firing and telemetry data is actively transmitted.
4. **Finalized:** Graceful teardown via `on_cleanup()` and `on_shutdown()`.

This architecture allows a central "Mission Supervisor" node to configure all UAV subsystems, verify sensor integrity, and only issue the activate command when the entire stack is healthy.

6 Real-Time Execution: PREEMPT_RT

While Zero-Copy IPC solves memory bottlenecks, CPU scheduling remains a challenge. A standard Linux kernel uses completely fair scheduling (CFS), which balances fairness over determinism. For a 400Hz quadcopter PID loop, a scheduling jitter of 3 milliseconds can destabilize the aircraft.

To achieve hard real-time execution in ROS 2:

- **PREEMPT_RT Kernel:** The OS must be patched, making spinlocks preemptible and eliminating bounded priority inversion.
- **Memory Locking:** Page faults must be prevented using memory locking (`mlockall`) to lock the node's memory into physical RAM, bypassing the OS swap disk.
- **Thread Priorities (`SCHED_FIFO`):** The ROS 2 Executor thread running the flight controller must be elevated to a real-time POSIX priority, ensuring it preempts all other OS background tasks the microsecond a timer triggers.

7 Implementation in Modern C++

When instantiating a publisher in C++, QoS profiles are passed via the `rclcpp::QoS` class. A production-grade sensor node configures its profile explicitly:

```cpp
#include "rclcpp/rclcpp.hpp"
#include "uav_interfaces/msg/drone_telemetry.hpp"

class TelemetryNode : public rclcpp::Node {
public:
    TelemetryNode() : Node("telemetry_node") {
        // Construct a highly-specific QoS profile for RF telemetry
        rclcpp::QoS telemetry_qos_profile(10); // Keep_last with depth 10
        telemetry_qos_profile.best_effort();
        telemetry_qos_profile.durability_volatile();
        telemetry_qos_profile.deadline(std::chrono::milliseconds(500));

        publisher_ = this->create_publisher<uav_interfaces::msg::DroneTelemetry>(
            "/uav/telemetry",
            telemetry_qos_profile
        );
    }
private:
    rclcpp::Publisher<uav_interfaces::msg::DroneTelemetry>::SharedPtr publisher_;
};
```

Mastering Custom Interfaces, Lifecycle orchestration, and QoS is what separates hobbyist robotics from aviation-grade software engineering.